

TALLER PRÁCTICO – BLOQUE 5

De clases aisladas a objetos que colaboran

Interfaces · Asociación · Agregación · Composición · Herencia · Componentes UML

Dato	Información
Caso conductor	SmartLibrary
Duración	120 minutos
Modalidad	Parejas o equipos de 3 estudiantes
Herramientas	Draw.io / Visual Paradigm + Java (IDE de preferencia)
Producto	Diagrama UML actualizado + implementación Java parcial + justificación de decisiones
Medio de entrega	Repositorio de git

Marco conceptual aplicado en todo el taller

Antes de resolver cada actividad se fijó el criterio de decisión que se usa de forma uniforme, para que ninguna relación se elija por intuición ni por el nombre de las clases.

Relación	Qué significa	Pregunta que la identifica
Asociación	Dos objetos se conocen y colaboran. Ninguno es parte del otro y cada uno tiene ciclo de vida propio.	¿Solo necesito expresar que se conocen para trabajar juntos?
Agregación	Existe una relación todo–parte, pero la parte sobrevive sin el todo e incluso puede pasar a otro todo.	Si destruyo el todo, ¿la parte sigue teniendo significado propio? Sí.
Composición	Relación todo–parte donde la parte pertenece a un único todo y no tiene identidad ni sentido sin él.	Si destruyo el todo, ¿la parte sigue teniendo significado propio? No.

Herencia	Generalización «es-un»: la subclase puede sustituir a la superclase en cualquier contexto donde ésta se use.	¿Puedo reemplazar el objeto padre por el hijo sin romper el significado?
Interfaz	Contrato de capacidad: promete que el objeto sabe hacer algo, sin decir qué es ni cómo lo hace.	¿Es una capacidad opcional que solo algunas clases tienen?

Regla de trabajo del taller: no se asume que toda relación «tiene-un» es composición, ni que compartir atributos justifica herencia. Cada decisión de este documento se sustenta citando el requisito (R10 a R14) que la respalda.

Actividad 1 – Diagnóstico de relaciones

En esta actividad todavía no se dibujan símbolos UML. Se describe en lenguaje natural qué significa cada relación dentro del problema y si los objetos pueden existir de forma independiente.

Elementos	¿Qué relación existe en el problema?	¿Existencia independiente?	Decisión inicial
Estudiante – Préstamo	El estudiante solicita préstamos. El préstamo registra quién retiró el ejemplar, pero no forma parte del estudiante ni lo contiene.	Sí, ambos. El estudiante está registrado aunque nunca pida un libro, y el préstamo debe conservarse como historial aunque el estudiante se gradúe.	Asociación 1 → 0..*
Libro – Ejemplar	El libro es la obra catalogada (título, autor, ISBN); el ejemplar es la copia física que se presta. R10 establece que cada ejemplar corresponde a un <i>único</i> libro.	No. El ejemplar aislado carece de identidad: no se sabría qué obra representa, y no puede reasignarse a otro libro.	Composición 1 ◆→ 0..* (supuesto declarado en la Act. 2)
Préstamo – Renovación	R11: el préstamo registra las renovaciones realizadas. Cada renovación conserva la fecha anterior de devolución, la nueva fecha y la fecha en que se renovó.	No. La renovación es un registro interno del préstamo; sus tres fechas solo significan algo referidas a ese préstamo concreto.	Composición 1 ◆→ 0..*
Usuario – Estudiante	El estudiante es un tipo de usuario de la biblioteca: comparte identificación, nombre y correo (R12) y añade código estudiantil y programa académico.	No aplica: no es una relación todo–parte sino una generalización. El estudiante es un usuario, no <i>tiene</i> un usuario.	Herencia («es-un») a validar en la Act. 3
Usuario – Bibliotecario	El bibliotecario también es un usuario del sistema, con código de empleado y turno como información particular (R12).	Igual que el caso anterior: relación «es-un», no «tiene-un».	Herencia («es-un») a validar en la Act. 3

Preguntas de discusión

1. ¿Que dos clases se relacionen significa necesariamente que una sea parte de la otra?

No. Relacionarse significa únicamente que los objetos necesitan conocerse para colaborar. Estudiante y Prestamo se relacionan porque el préstamo debe saber quién retiró el ejemplar, pero el préstamo no es una pieza del estudiante ni al revés. En un sistema real la mayoría de relaciones son asociaciones simples; el todo–parte es la excepción y debe demostrarse, no suponerse.

2. ¿Qué evidencia del requisito utilizaron para decidir?

- **R10** («cada ejemplar corresponde a un *único* libro»): la palabra *único* indica pertenencia exclusiva y no compartida, que es la marca de la composición.
- **R11** («un préstamo registra las renovaciones realizadas»): la renovación nace dentro del préstamo, como parte de su historial interno.
- **R12** («estudiantes y bibliotecarios comparten identificación, nombre y correo, pero poseen información particular»): sugiere generalización, aunque debe verificarse con la prueba «es-un» y no aceptarse solo por los atributos repetidos.
- **R13** («*algunos* usuarios pueden recibir notificaciones»): la palabra *algunos* demuestra que notificar es una capacidad opcional, por lo tanto una interfaz y no un atributo de Usuario.
- **R14** (organización en cuatro áreas funcionales): sustenta la vista de componentes de la Actividad 6.

3. ¿Cuál de las relaciones anteriores sería incorrecto decidir únicamente mirando los nombres de las clases?

Usuario – Estudiante y **Usuario – Bibliotecario**. Los nombres sugieren de inmediato una jerarquía y empujan a colocar herencia sin analizar nada. Pero la herencia no se justifica porque los nombres suenan jerárquicos ni porque se repitan atributos: se justifica porque la subclase se comporta como la superclase en todos los contextos donde ésta se utiliza. Esa verificación es el objeto de la Actividad 3.

El segundo caso riesgoso es **Libro – Ejemplar**: los nombres parecen sinónimos y muchos equipos los fusionan en una sola clase, perdiendo la distinción entre la obra del catálogo y la copia física que efectivamente se presta, que es justamente lo que R10 exige separar.

Actividad 2 – Asociación, agregación y composición

Fragmento del diagrama de clases con Libro, Ejemplar, Estudiante, Prestamo y Renovacion, incluyendo multiplicidades y el tipo de relación seleccionado.

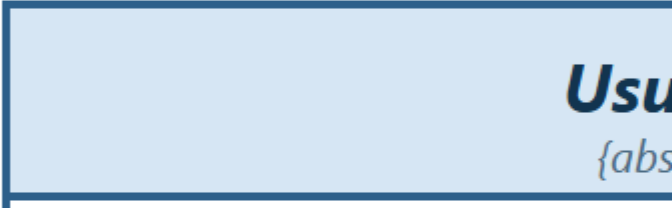
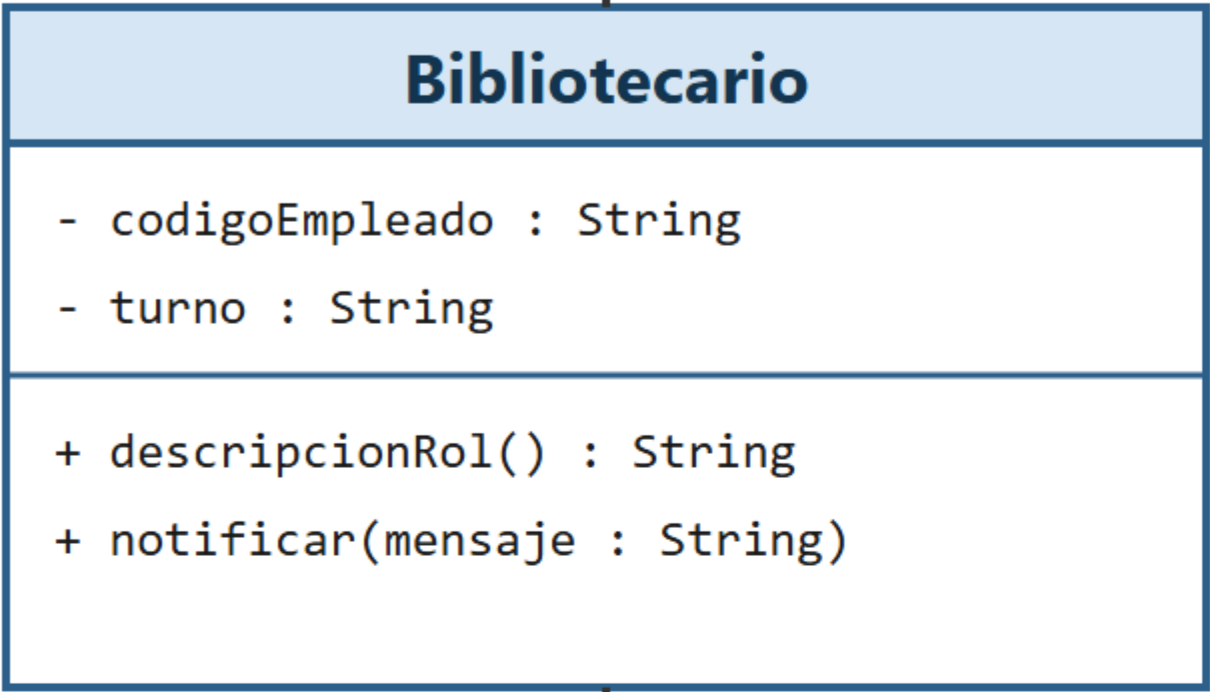
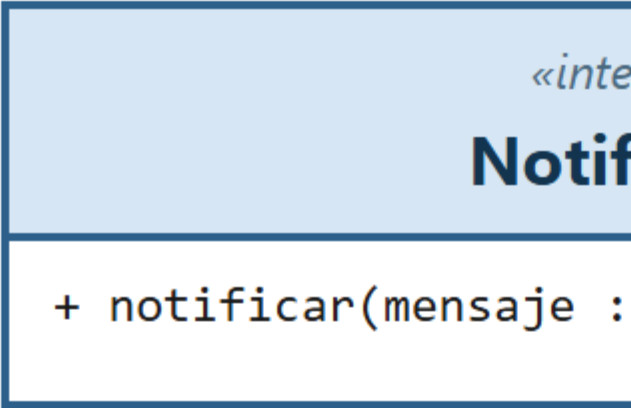


Figura 1. Diagrama de clases de SmartLibrary con las relaciones, multiplicidades, la jerarquía de usuarios y la realización de la interfaz Notificable.

Relación	Tipo seleccionado	Justificación basada en el dominio
Estudiante – Prestamo	Asociación 1 → 0..*	El préstamo necesita saber a qué estudiante pertenece, pero no es una parte suya. Ambos tienen ciclo de vida propio: el estudiante se registra antes de solicitar nada y el préstamo debe conservarse como historial institucional aunque el estudiante deje la universidad. Modelarlo como composición obligaría a eliminar todo el historial al dar de baja a un estudiante, lo que contradice el funcionamiento de una biblioteca.
Prestamo – Ejemplar	Asociación 0..* → 1	El préstamo usa temporalmente un ejemplar; no lo contiene. El ejemplar existía en la estantería antes del préstamo y sigue existiendo después de la devolución, disponible para un préstamo posterior. Que un objeto sea utilizado por otro durante un periodo no lo convierte en parte de él: la relación expresa colaboración, no pertenencia.
Libro – Ejemplar	Composición 1 ♦ → 0..*	R10 establece que cada ejemplar corresponde a un único libro y no puede reasignarse a otro. El ejemplar es la materialización física de la obra: sin el libro que lo define no tiene identidad dentro del sistema, porque no se sabría qué obra representa. Si el libro se retira del catálogo, sus ejemplares dejan de tener significado como registros. Se cumplen las dos condiciones de la composición: pertenencia exclusiva y dependencia de significado.
Prestamo – Renovacion	Composición 1 ♦ → 0..*	R11 define la renovación como un registro interno del préstamo, formado por la fecha anterior de devolución, la nueva fecha y la fecha en que se realizó. Esos tres datos solo tienen sentido referidos a un préstamo concreto. Una renovación nunca se crea suelta, nunca se consulta por sí sola y nunca se transfiere a otro préstamo; si el préstamo desaparece, la renovación deja de significar algo. Es el caso más claro de composición del modelo.

Aplicación de las preguntas-criterio

Las cuatro preguntas del enunciado no se usaron como fórmula automática sino como un filtro aplicado en orden, deteniéndose en la primera que resulta afirmativa.

1. **¿Solo necesito expresar que los objetos se conocen o colaboran?** Si la respuesta es sí, la decisión termina ahí: es asociación. Es el caso de **Estudiante-Prestamo** y **Prestamo-Ejemplar**.
2. **¿Existe una relación conceptual todo–parte?** Solo se evalúa si la anterior fue negativa. **Libro-Ejemplar** y **Prestamo-Renovacion** sí la presentan.
3. **¿La parte tiene sentido y ciclo de vida independiente?** Si la respuesta es sí, la relación es agregación. En los dos casos anteriores la respuesta es **no**.
4. **¿La eliminación conceptual del todo implica que la parte deja de tener significado?** La respuesta afirmativa confirma la composición en ambos casos.

Reto de razonamiento: supuesto documentado

El enunciado deja abierta una regla en **Libro – Ejemplar**, por lo que otro equipo puede llegar a una decisión distinta igualmente válida. Se deja explícito el supuesto que hace correcta la decisión adoptada.

Supuesto adoptado: el **Ejemplar** se trata como una copia registrada que existe únicamente en función de la obra catalogada; SmartLibrary no mantiene ejemplares huérfanos sin libro asociado. Bajo este supuesto la relación es **composición**.

Alternativa válida: si se asumiera que la biblioteca puede conservar un ejemplar físico mientras se corrige o reemplaza su ficha de catálogo —por ejemplo, un ejemplar donado que aún no tiene registro—, el ejemplar tendría ciclo de vida propio y la relación correcta pasaría a ser **agregación** (**Libro ◇– Ejemplar**). El símbolo no es correcto por sí mismo: lo es respecto de la regla de negocio declarada.

Actividad 3 – Herencia: ¿es-un o solo se parecen?

Se evalúa la propuesta de generalización `Usuario` → `Estudiante`, `Bibliotecario` aplicando la prueba de sustitución: la herencia es válida solo si un objeto de la subclase puede ocupar el lugar de la superclase en todo contexto donde ésta se use, sin romper el significado del modelo.

4. ¿Estudiante es realmente un tipo de Usuario dentro de SmartLibrary? Justifique.

Sí. El estudiante cumple la prueba de sustitución: en cualquier operación donde SmartLibrary requiera un usuario —identificarse ante el sistema, ser destinatario de un préstamo, aparecer en un reporte de deudores— un estudiante puede ocupar ese lugar sin que el significado del modelo se rompa. No hereda únicamente `identificacion`, `nombre` y `correo`: hereda el **rol de usuario de la biblioteca**. Sus atributos propios, `codigoEstudiantil` y `programaAcademico`, especializan ese rol sin contradecirlo.

5. ¿Bibliotecario es realmente un tipo de Usuario? Justifique.

Sí, con una precisión importante. El bibliotecario es un usuario *del sistema*: se identifica, tiene nombre y correo institucional y puede recibir notificaciones. La generalización es válida siempre que `Usuario` se defina como «persona identificada ante SmartLibrary» y no como «persona que toma libros en préstamo». Si `Usuario` se definiera de la segunda manera, el bibliotecario dejaría de ser sustituible —porque no participa como solicitante en los préstamos— y la herencia quedaría invalidada.

Definición adoptada: `Usuario` es la abstracción de toda persona identificada ante SmartLibrary, con datos de contacto y un rol dentro del sistema. Con esta definición ambas especializaciones satisfacen la relación «es-un».

6. ¿La existencia de `identificacion`, `nombre` y `correo` repetidos sería suficiente para crear la superclase `Usuario`? ¿Por qué?

No, no sería suficiente. La repetición de atributos es un indicio, nunca una justificación. La herencia es una afirmación conceptual sobre la naturaleza de los objetos («todo estudiante es un usuario»), no un mecanismo para ahorrar líneas de código. Si la superclase se creara solo para no repetir tres campos, se estaría usando una relación semántica fuerte para resolver un problema de duplicación, que es un problema distinto y tiene otras soluciones.

La superclase `Usuario` se justifica porque cumple tres condiciones simultáneas, y ninguna de ellas es la repetición de atributos:

- **Representa una abstracción real del dominio:** el concepto «usuario de la biblioteca» existe en el negocio, no es un invento del programador.
- **Cumple la sustitución:** donde el sistema espera un usuario, un estudiante o un bibliotecario funcionan correctamente.
- **Unifica comportamiento, no solo datos:** ambos se identifican ante el sistema y responden a la pregunta «qué rol cumple esta persona».

7. Propongan al menos una situación en la que crear una superclase únicamente para evitar atributos repetidos produciría un diseño conceptualmente débil.

Se presentan dos situaciones dentro del propio dominio de SmartLibrary.

Situación A — la superclase artificial `EntidadRegistrable`. `Libro` tiene `isbn` y `titulo`; `Estudiante` tiene `codigoEstudiantil` y `nombre`; `Ejemplar` tiene `codigoBarras`. Los tres poseen «un código y una denominación», y sería técnicamente posible extraer una superclase `EntidadRegistrable` con los campos `codigo` y `descripcion`. El diseño resultante sería falso: un libro no es un tipo de la misma cosa que un estudiante. La jerarquía no expresaría ninguna verdad del dominio y provocaría que cualquier método declarado en la superclase tuviera que aceptar objetos sin nada en común, obligando a verificar el tipo real antes de operar.

Situación B — la superclase `RegistroConFechas`. `Prestamo` y `Renovacion` manejan fechas, y también las manejaría una futura clase `Reserva`. Crear una superclase común solo por ese motivo uniría conceptos con reglas incompatibles: la fecha de un préstamo puede modificarse al renovar, la de una renovación es un hecho histórico inmutable y la de una reserva expira automáticamente. Una jerarquía basada en «tienen fechas» obligaría a que una clase heredara comportamiento que debe rechazar, y romper el contrato de la superclase es exactamente lo que una herencia mal justificada produce.

Decisión final sobre la generalización

La generalización se considera **válida** y el diagrama se actualiza con `Usuario` como clase abstracta. Se declara abstracta porque en SmartLibrary no existe ninguna persona que sea «solo usuario»: toda persona real es estudiante o bibliotecario, de modo que instanciar `Usuario` directamente no representaría nada del dominio.

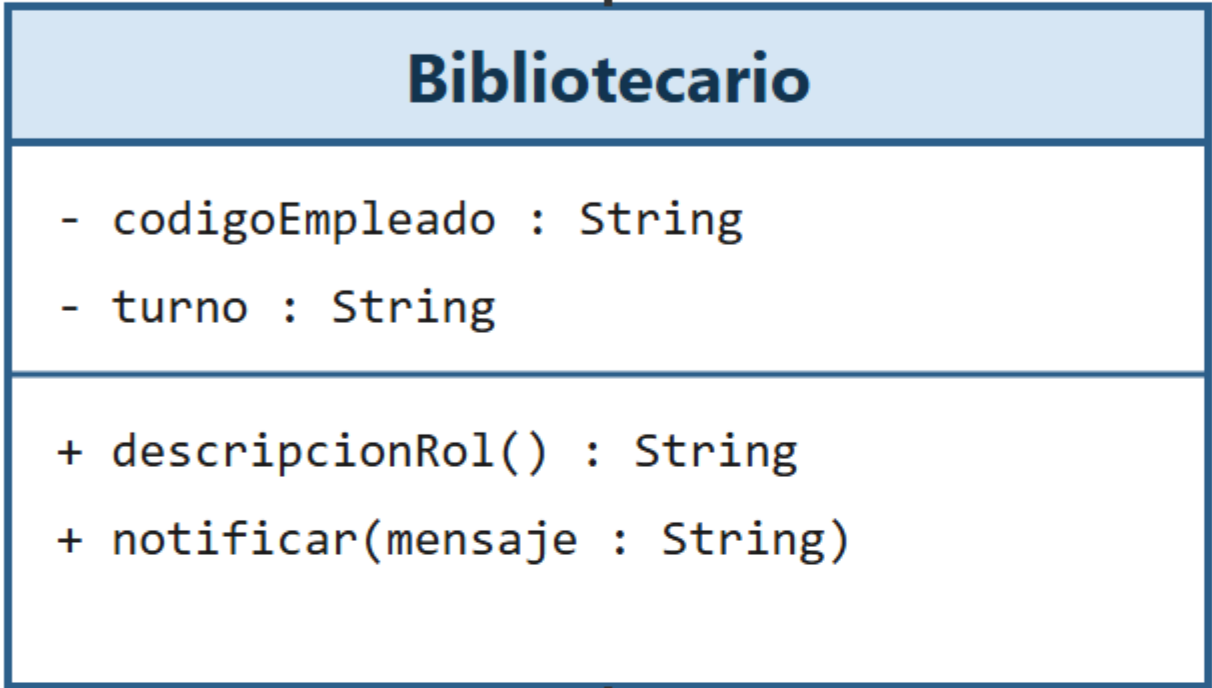
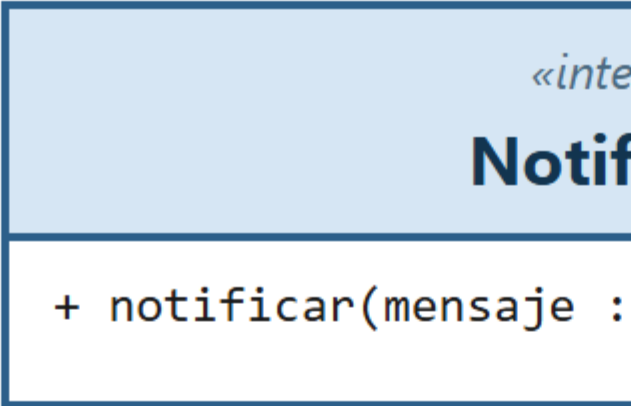


Figura 2. Jerarquía de usuarios y contrato Notificable, con la justificación de la generalización y de por qué `notificar()` no se declara en Usuario.

Actividad 4 – Interfaces: expresar un contrato

```
public interface Notificable {  
    void notificar(String mensaje);  
}
```

8. ¿Qué clase o clases deberían implementar Notificable y por qué?

Implementan `Notificable` las clases `Estudiante` y `Bibliotecario`. La decisión se apoya directamente en R13: «*algunos* usuarios pueden recibir notificaciones del sistema». La palabra *algunos* es determinante, porque indica que recibir notificaciones no es una característica inherente a ser usuario sino una capacidad que algunas especializaciones poseen.

Por esa razón el método `notificar` **no** se coloca en la clase `Usuario`. Si se declarara allí, el modelo afirmarí que todo usuario, sin excepción, es notificable, lo que contradice el requisito y obligaría a que cualquier futuro tipo de usuario no notificable —por ejemplo un usuario visitante de consulta en sala— heredara un método vacío o que lanza error. La interfaz permite que la capacidad se añada exactamente donde corresponde.

Además, cada clase implementa el contrato de forma distinta y esa diferencia es intencional: el estudiante se notifica por correo electrónico y el bibliotecario por el panel interno del sistema. Ambos cumplen la misma promesa con medios diferentes, que es precisamente lo que una interfaz permite expresar.

9. Representación de la interfaz y la relación de realización en UML

La representación UML de la interfaz y de la relación de realización corresponde a la **Figura 2**. Allí se aprecia la diferencia gráfica que importa: la **herencia** hacia `Usuario` se dibuja con línea continua y la **realización** de `Notificable` con línea punteada, ambas con triángulo vacío. Una clase tiene una sola línea continua hacia arriba y puede tener varias punteadas.

La diferencia gráfica importa: la **herencia** se dibuja con línea continua y la **realización** de una interfaz con línea punteada, ambas con triángulo vacío. Una clase tiene una sola línea continua hacia arriba y puede tener varias punteadas.

10. ¿Qué garantiza el contrato Notificable y qué NO especifica?

Lo que garantiza	Lo que NO especifica
- Que el objeto sabe recibir un mensaje de notificación. - La firma exacta del método: nombre <code>notificar</code>, un parámetro <code>String</code> y sin valor de retorno. - Que cualquier objeto declarado como <code>Notificable</code> puede recibir la llamada sin conocer su clase real.	Cómo se envía el mensaje: correo, SMS, consola o panel interno. Cuándo se envía ni quién dispara la notificación. Qué es el objeto que la implementa: no dice que sea una persona ni un usuario. - Qué ocurre si el envío falla ni si el mensaje se almacena.

11. ¿Por qué una interfaz no equivale a una superclase?

Porque responden a preguntas distintas. La superclase responde «¿qué es este objeto?» y la interfaz responde «¿qué sabe hacer este objeto?». Un estudiante es un usuario —eso define su naturaleza— y además *sabe* notificarse —eso es una capacidad que podría no tener sin dejar de ser estudiante.

Las diferencias técnicas se derivan de esa distinción conceptual:

Aspecto	Superclase (Usuario)	Interfaz (Notificable)
Pregunta que responde	¿Qué es el objeto?	¿Qué sabe hacer el objeto?
Aporta estado	Sí: hereda identificación, nombre y correo.	No: solo declara métodos, no atributos de instancia.
Cuántas se admiten	Una sola: Java no permite herencia múltiple de clases.	Varias a la vez sobre la misma clase.
Alcance	Solo dentro de la jerarquía.	Clases de jerarquías totalmente distintas pueden implementarla.

Tabla de respuestas del equipo

Pregunta	Respuesta del equipo
¿Qué promete una clase que implementa Notificable?	Promete que sabe recibir un mensaje de notificación a través del método <code>notificar(String mensaje)</code> . Es una promesa de capacidad : quien tenga una referencia de tipo <code>Notificable</code> puede invocar ese método con la certeza de que existe una implementación real, sin necesidad de conocer la clase concreta del objeto.
¿La interfaz define cómo se envía el mensaje?	No. La interfaz define únicamente la firma del método. El medio de envío es decisión de cada clase que la implementa: <code>Estudiante</code> lo hace por correo electrónico y <code>Bibliotecario</code> por el panel interno del sistema. Esa separación entre <i>qué se promete</i> y <i>cómo se cumple</i> es el valor central de una interfaz.
¿Puede una clase heredar de Usuario e implementar Notificable al mismo tiempo? Explique.	Sí, y es exactamente lo que hace el diseño. Java permite extender una sola clase pero implementar varias interfaces: <code>public class Estudiante extends Usuario implements Notificable</code> . No hay conflicto porque las dos relaciones

	expresan cosas diferentes: <code>extends</code> Usuario declara <i>qué es</i> el objeto y de dónde hereda su estado; <code>implements</code> Notificable declara <i>qué sabe hacer</i> adicionalmente. Un estudiante es un usuario que, además, es notificable.
--	---

Actividad 5 – Implementación representativa en Java

Se implementa únicamente el fragmento necesario para validar las decisiones de diseño. No se construye una aplicación completa: no hay interfaz gráfica, persistencia ni menús.

5.1 Usuario y especializaciones

Notificable.java

```
public interface Notificable {  
    void notificar(String mensaje);  
}
```

Usuario.java

```
public abstract class Usuario {  
  
    private String identificacion;  
    private String nombre;  
    private String correo;  
  
    public Usuario(String identificacion, String nombre, String correo) {  
        this.identificacion = identificacion;  
        this.nombre = nombre;  
        this.correo = correo;  
    }  
  
    public String getIdentificacion() {  
        return identificacion;  
    }  
  
    public String getNombre() {  
        return nombre;  
    }  
  
    public String getCorreo() {  
        return correo;  
    }  
  
    // Cada especializacion declara su rol: obliga a que toda subclase  
    // aporte significado propio y no solo reutilice atributos.  
    public abstract String descripcionRol();  
}
```

```
}
```

Por qué es abstract: en SmartLibrary no existe una persona que sea «solo usuario», por lo que la clase no debe poder instanciarse. El método abstracto `descripcionRol()` refuerza la decisión de la Actividad 3: obliga a cada subclase a declarar qué es realmente, evitando que la jerarquía se reduzca a compartir tres campos.

Estudiante.java

```
public class Estudiante extends Usuario implements Notificable {

    private String codigoEstudiantil;
    private String programaAcademico;

    public Estudiante(String identificacion, String nombre, String correo,
                      String codigoEstudiantil, String programaAcademico) {
        super(identificacion, nombre, correo);           // datos comunes de Usuario
        this.codigoEstudiantil = codigoEstudiantil;      // datos propios
        this.programaAcademico = programaAcademico;
    }

    public String getCodigoEstudiantil() {
        return codigoEstudiantil;
    }

    public String getProgramaAcademico() {
        return programaAcademico;
    }

    @Override
    public String descripcionRol() {
        return "Estudiante del programa " + programaAcademico;
    }

    @Override
    public void notificar(String mensaje) {
        // El contrato solo exige recibir el mensaje; el medio lo decide la clase.
        System.out.println("[CORREO -> " + getCorreo() + "] " + mensaje);
    }
}
```

Bibliotecario.java

```
public class Bibliotecario extends Usuario implements Notificable {

    private String codigoEmpleado;
    private String turno;

    public Bibliotecario(String identificacion, String nombre, String correo,
                        String codigoEmpleado, String turno) {
        super(identificacion, nombre, correo);
        this.codigoEmpleado = codigoEmpleado;
        this.turno = turno;
    }

    public String getCodigoEmpleado() {
        return codigoEmpleado;
    }

    public String getTurno() {
        return turno;
    }

    @Override
    public String descripcionRol() {
        return "Bibliotecario del turno " + turno;
    }

    @Override
    public void notificar(String mensaje) {
        // Misma promesa, medio distinto: demuestra que la interfaz no impone el como.
        System.out.println("[PANEL INTERNO - empleado " + codigoEmpleado + "] " + mensaje);
    }
}
```

Libro.java y Ejemplar.java (composición de la Actividad 2)

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Libro {
```

```

private String isbn;
private String titulo;
private String autor;
private final List<Ejemplar> ejemplares = new ArrayList<>();

public Libro(String isbn, String titulo, String autor) {
    this.isbn = isbn;
    this.titulo = titulo;
    this.autor = autor;
}

// La COMPOSICION se hace visible aqui: el ejemplar solo puede nacer
// a traves del libro que lo contiene. Nadie puede crear un Ejemplar suelto.
public Ejemplar registrarEjemplar(String codigoBarras) {
    Ejemplar ejemplar = new Ejemplar(codigoBarras, this);
    ejemplares.add(ejemplar);
    return ejemplar;
}

public List<Ejemplar> getEjemplares() {
    return Collections.unmodifiableList(ejemplares); // nadie altera las partes
}

public String getTitulo() {
    return titulo;
}
}

public class Ejemplar {

    private String codigoBarras;
    private boolean disponible;
    private final Libro libro; // final: nunca se reasigna a otro libro (R10)

    // Constructor de paquete, no publico: refuerza la composicion.
    Ejemplar(String codigoBarras, Libro libro) {
        this.codigoBarras = codigoBarras;
        this.libro = libro;
        this.disponible = true;
    }

    public String getCodigoBarras() {

```

```

        return codigoBarras;
    }

    public Libro getLibro() {
        return libro;
    }

    public boolean estaDisponible() {
        return disponible;
    }

    public void marcarPrestado() {
        this.disponible = false;
    }

    public void marcarDisponible() {
        this.disponible = true;
    }
}

```

Cómo el código refleja el UML: la composición Libro ◆— Ejemplar no se expresa solo con una lista. Se expresa con tres decisiones combinadas: el Ejemplar no tiene constructor público, solo nace desde registrarEjemplar; su referencia al libro es final, de modo que no puede reasignarse; y la lista se devuelve inmodificable para que nadie inserte o elimine partes desde fuera del todo.

5.2 Préstamo y renovaciones

Renovacion.java

```

import java.time.LocalDate;

public class Renovacion {

    private final LocalDate fechaRenovacion;    // cuando se realizo el tramite
    private final LocalDate fechaAnterior;      // fecha de devolucion que tenia antes
    private final LocalDate nuevaFecha;         // fecha de devolucion que queda vigente

    // Constructor de paquete: una Renovacion solo puede crearla el Prestamo.
    Renovacion(LocalDate fechaAnterior, LocalDate nuevaFecha, LocalDate fechaRenovacion) {
        this.fechaAnterior = fechaAnterior;
        this.nuevaFecha = nuevaFecha;
        this.fechaRenovacion = fechaRenovacion;
    }
}

```

```

    public LocalDate getFechaRenovacion() {
        return fechaRenovacion;
    }

    public LocalDate getFechaAnterior() {
        return fechaAnterior;
    }

    public LocalDate getNuevaFecha() {
        return nuevaFecha;
    }

    @Override
    public String toString() {
        return "Renovacion del " + fechaRenovacion
            + ": " + fechaAnterior + " -> " + nuevaFecha;
    }
}

```

Los tres atributos son `final` y no existen métodos `set`: una renovación es un **hecho histórico** y, una vez registrada, no debe poder alterarse. Esto cumple R11, que exige conservar la fecha anterior.

Prestamo.java

```

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Prestamo {

    private final Estudiante estudiante;           // ASOCIACION
    private final Ejemplar ejemplar;              // ASOCIACION
    private final LocalDate fechaPrestamo;
    private LocalDate fechaPrevistaDevolucion;     // cambia al renovar
    private final List<Renovacion> renovaciones = new ArrayList<>(); // COMPOSICION

    public static final int MAXIMO_RENOVACIONES = 2;

    public Prestamo(Estudiante estudiante, Ejemplar ejemplar,

```

```

        LocalDate fechaPrestamo, LocalDate fechaPrevistaDevolucion) {
    this.estudiante = estudiante;
    this.ejemplar = ejemplar;
    this.fechaPrestamo = fechaPrestamo;
    this.fechaPrevistaDevolucion = fechaPrevistaDevolucion;
    ejemplar.marcarPrestado();
}

public void renovar(LocalDate nuevaFecha) {

    // 1. Validar que nuevaFecha sea posterior a la fecha actual prevista.
    if (nuevaFecha == null || !nuevaFecha.isAfter(fechaPrevistaDevolucion)) {
        throw new IllegalArgumentException(
            "Renovacion invalida: la nueva fecha (" + nuevaFecha + ") debe ser "
            + "posterior a la fecha prevista actual (" + fechaPrevistaDevolucion + ").");
    }

    if (renovaciones.size() >= MAXIMO_RENOVACIONES) {
        throw new IllegalStateException(
            "Renovacion invalida: el prestamo ya alcanzo el maximo de "
            + MAXIMO_RENOVACIONES + " renovaciones.");
    }

    // 2. Crear la Renovacion conservando la fecha anterior (R11).
    Renovacion renovacion =
        new Renovacion(fechaPrevistaDevolucion, nuevaFecha, LocalDate.now());

    // 3. Almacenarla dentro del prestamo.
    renovaciones.add(renovacion);

    // 4. Actualizar la fecha prevista de devolucion.
    this.fechaPrevistaDevolucion = nuevaFecha;
}

public List<Renovacion> getRenovaciones() {
    return Collections.unmodifiableList(renovaciones);
}

public int cantidadRenovaciones() {
    return renovaciones.size();
}

```

```

    public LocalDate getFechaPrevistaDevolucion() {
        return fechaPrevistaDevolucion;
    }

    public Estudiante getEstudiante() {
        return estudiante;
    }

    public Ejemplar getEjemplar() {
        return ejemplar;
    }
}

```

Cómo el código protege la regla de renovación: el método renovar es el **único** camino para crear una Renovacion, porque su constructor no es público. Además, fechaPrevistaDevolucion no tiene método set: solo se modifica dentro de renovar, después de validar. Así es imposible que el sistema quede con una fecha nueva sin su renovación registrada, o con una renovación sin que la fecha se haya actualizado.

5.3 Prueba mínima

PruebaSmartLibrary.java

```

import java.time.LocalDate;

public class PruebaSmartLibrary {

    public static void main(String[] args) {

        // --- Montaje del escenario ---
        Estudiante estudiante = new Estudiante(
            "1094567890", "Camilo Restrepo", "camilo@universidad.edu.co",
            "EST-2024-118", "Ingenieria de Sistemas");

        Bibliotecario bibliotecario = new Bibliotecario(
            "43876211", "Marta Ochoa", "marta@biblioteca.edu.co",
            "EMP-045", "Manana");

        Libro libro = new Libro("978-0132350884", "Clean Code", "Robert C. Martin");
        Ejemplar ejemplar = libro.registrarEjemplar("EJ-00123");

        Prestamo prestamo = new Prestamo(

```

```

        estudiante, ejemplar,
        LocalDate.of(2026, 9, 21),      // fecha del prestamo
        LocalDate.of(2026, 10, 5));    // fecha prevista de devolucion

    System.out.println("=== PRUEBA 1: renovacion VALIDA ===");
    System.out.println("Fecha prevista inicial : " +
        prestamo.getFechaPrevistaDevolucion());

    prestamo.renovar(LocalDate.of(2026, 10, 15));
    estudiante.notificar("Su prestamo de '" + libro.getTitulo() + "' fue renovado.");
    bibliotecario.notificar("Renovacion registrada para el ejemplar "
        + ejemplar.getCodigoBarras() + ".");

    System.out.println("Nueva fecha prevista : " +
        prestamo.getFechaPrevistaDevolucion());
    System.out.println("Cantidad renovaciones : " + prestamo.cantidadRenovaciones());
    System.out.println("Historial : " + prestamo.getRenovaciones().get(0));

    System.out.println();
    System.out.println("=== PRUEBA 2: renovacion INVALIDA ===");
    try {
        // Se intenta renovar hacia una fecha ANTERIOR a la vigente.
        prestamo.renovar(LocalDate.of(2026, 10, 1));
        System.out.println("ERROR: la renovacion invalida fue aceptada.");
    } catch (IllegalArgumentException e) {
        System.out.println("Rechazada correctamente -> " + e.getMessage());
    }

    System.out.println("Fecha prevista tras el intento : "
        + prestamo.getFechaPrevistaDevolucion());
    System.out.println("Cantidad renovaciones : "
        + prestamo.cantidadRenovaciones());
}
}

```

Evidencia de ejecución

```

=== PRUEBA 1: renovacion VALIDA ===
Fecha prevista inicial : 2026-10-05
[CORREO -> camilo@universidad.edu.co] Su prestamo de 'Clean Code' fue renovado.
[PANEL INTERNO - empleado EMP-045] Renovacion registrada para el ejemplar EJ-00123.
Nueva fecha prevista : 2026-10-15

```

Cantidad renovaciones : 1
Historial : Renovacion del 2026-09-30: 2026-10-05 -> 2026-10-15

=== PRUEBA 2: renovacion INVALIDA ===

Rechazada correctamente -> Renovacion invalida: la nueva fecha (2026-10-01) debe ser posterior a la fecha prevista actual (2026-10-15).

Fecha prevista tras el intento : 2026-10-15

Cantidad renovaciones : 1

Documentación de la renovación inválida: qué debería ocurrir y qué ocurre

Aspecto	Comportamiento esperado y verificado
Resultado de la operación	La renovación se rechaza lanzando <code>IllegalArgumentException</code> con un mensaje que explica la causa concreta.
Estado del préstamo	No cambia. La fecha prevista sigue siendo 2026-10-15, la misma que antes del intento.
Historial de renovaciones	No se contamina. Sigue habiendo una sola renovación: la válida. No queda registro de intentos fallidos.
Principio que se demuestra	La operación es atómica : o se cumplen todas las condiciones y el objeto avanza de forma consistente, o no ocurre absolutamente nada. El objeto nunca queda en un estado intermedio inválido.
Por qué la validación va dentro de Prestamo	Porque el préstamo es el único objeto que conoce su fecha vigente y su historial. Si la validación se hiciera en la clase de prueba o en una futura interfaz de usuario, cualquier otro punto del sistema podría renovar sin validar y la regla dejaría de estar garantizada.

Actividad 6 – Vista funcional mediante componentes UML

Vista funcional introductoria de SmartLibrary según R14. Es una agrupación por responsabilidad funcional, de alto nivel: no describe microservicios, infraestructura, contenedores, despliegue, APIs ni bases de datos.

Componente	Clases relacionadas	¿De qué otro componente necesita información? ¿Para qué?
Gestión de Usuarios	Usuario (abstracta), Estudiante, Bibliotecario, Notificable	De ningún otro componente: es el más independiente del modelo, ya que un usuario existe aunque nunca use la biblioteca. Los demás componentes dependen de él. Es quien ofrece la identificación de usuarios y la capacidad de notificación definida por el contrato Notificable .
Gestión de Catálogo	Libro, Ejemplar	Tampoco depende de otros componentes. Mantiene las obras y sus copias físicas, y ofrece a los demás la consulta de disponibilidad de ejemplares y el cambio de su estado (prestado / disponible).
Gestión de Préstamos	Prestamo, Renovacion	Necesita Gestión de Usuarios para validar que el estudiante solicitante está registrado y habilitado, y para notificarle renovaciones y vencimientos. Necesita Gestión de Catálogo para verificar que el ejemplar existe y está disponible, y para marcarlo como prestado o disponible al devolverlo.
Gestión de Reservas	Reserva (prevista del modelo de SmartLibrary)	Necesita Gestión de Usuarios para saber quién reserva y poder notificarle cuando el ejemplar quede libre. Necesita Gestión de Catálogo para conocer qué ejemplares existen de un libro. Necesita Gestión de Préstamos para saber si un ejemplar está actualmente prestado y cuándo se espera su devolución, ya que una reserva solo tiene sentido sobre material no disponible.

Diagrama de componentes

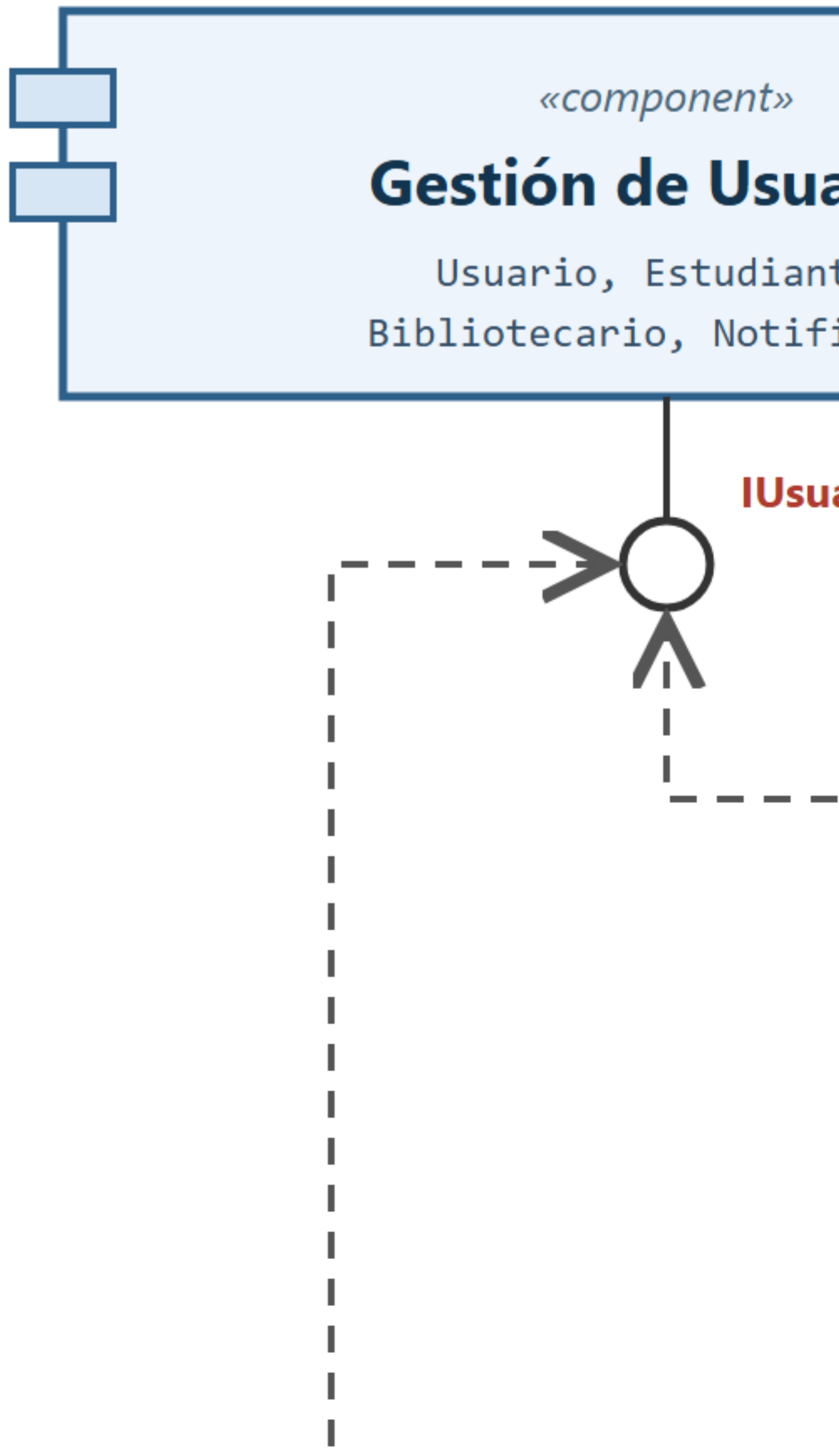
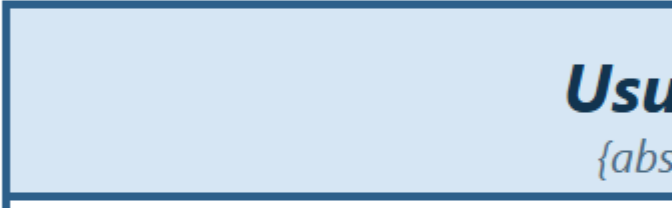
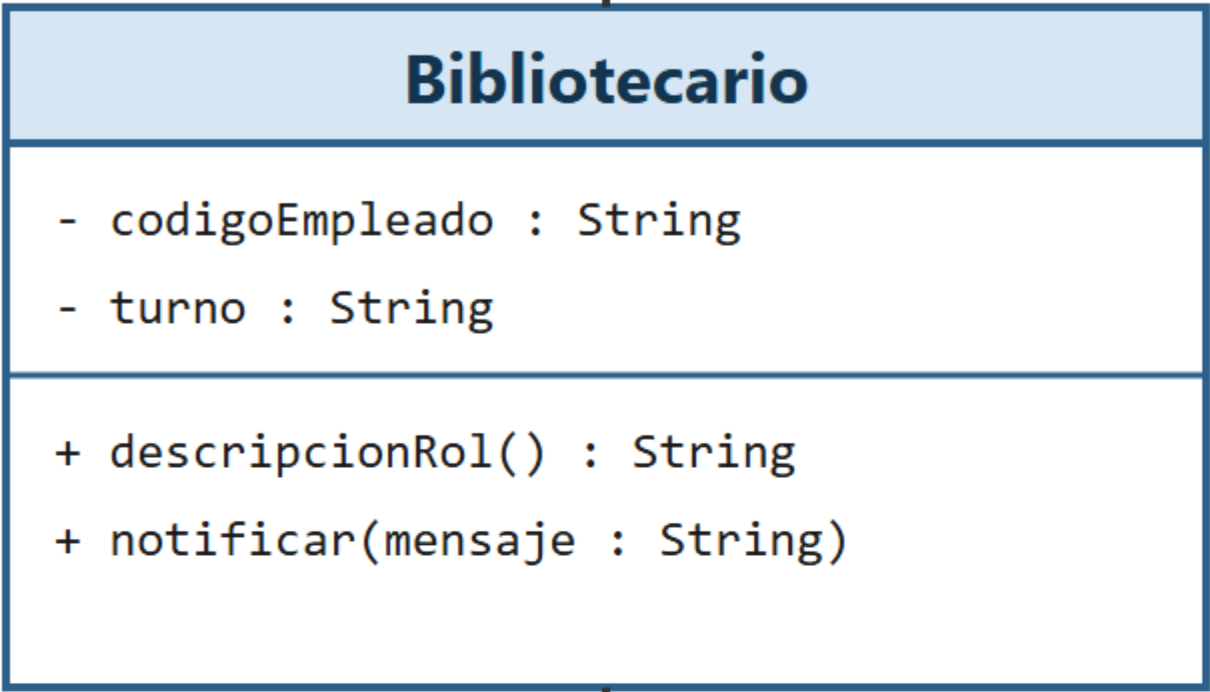
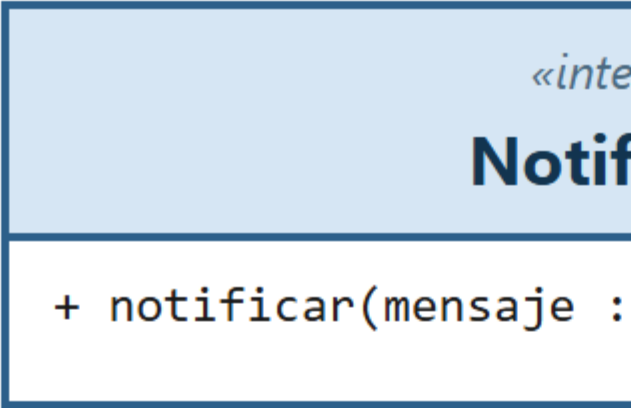


Figura 3. Vista funcional de SmartLibrary mediante componentes UML, con las interfaces provistas y requeridas de cada agrupación.

Diferencia entre una clase del diagrama de clases y un componente de la vista funcional: la clase describe la **estructura** de un objeto —sus atributos, sus operaciones y sus relaciones—, mientras que el componente describe una **agrupación funcional** del sistema por responsabilidad, y lo único que expone hacia afuera son sus interfaces. El componente oculta cuántas clases contiene y cómo se relacionan internamente; por eso puede cambiar por dentro sin afectar a los demás mientras respete lo que prometió ofrecer.

Diagrama de clases actualizado de SmartLibrary

Vista consolidada con todas las relaciones trabajadas en el taller.



Conclusión

«¿Qué cambia en el diseño cuando dejamos de mirar clases aisladas y empezamos a diseñar sus relaciones y contratos?»

Cambia el objeto mismo del diseño: se deja de describir *qué guarda* cada clase para decidir *de qué depende cada objeto y qué le promete a los demás*. Al aislarlas, cualquier clase parecía correcta con solo tener atributos coherentes; al relacionarlas, cada línea del diagrama se convierte en una afirmación sobre el dominio que debe sostenerse con un requisito. Elegir composición en lugar de asociación ya no es un detalle gráfico: determina que una renovación no pueda existir sin su préstamo y que el código deba impedirlo. La herencia deja de ser un atajo para no repetir atributos y pasa a comprometerse con que la subclase pueda sustituir a la superclase sin romper el significado. La interfaz separa por primera vez lo que un objeto promete de cómo lo cumple, permitiendo que estudiante y bibliotecario respondan igual por medios distintos. El diseño se vuelve verificable: cada decisión puede defenderse o refutarse citando una regla del problema.

(148 palabras)

Lista de verificación del estudiante

- No utilicé composición solo porque una clase contiene otra: en `Prestamo-Ejemplar` hay contención lógica y aun así se modeló como asociación, porque el ejemplar sobrevive al préstamo.
- Justifiqué las relaciones a partir del significado del dominio, citando R10, R11, R12, R13 y R14 en cada decisión.
- La herencia expresa «es-un» y no solamente reutilización de atributos: se aplicó la prueba de sustitución y se declaró la definición de `Usuario` que la hace válida.
- Puedo explicar la diferencia entre heredar una clase e implementar una interfaz: la superclase responde *qué es* el objeto, la interfaz *qué sabe hacer*.
- Mis multiplicidades son coherentes con los requisitos: `Libro 1 → 0..* Ejemplar` por R10 y `Prestamo 1 → 0..* Renovacion` por R11.
- El código refleja las relaciones mostradas en UML: la composición se hace cumplir con constructores no públicos, referencias `final` y colecciones inmodificables.
- `Prestamo` protege la regla de renovación: valida antes de modificar y la operación es atómica, comprobado con una renovación válida y una inválida.
- Mi diagrama de componentes muestra agrupaciones funcionales, no una arquitectura de microservicios ni infraestructura.
- No adelanté GRASP, SOLID ni patrones de diseño: el taller se limita a relaciones estructurales y contratos de comportamiento.

Criterios de evaluación y dónde se cubre cada uno

Criterio	Peso	Sección de este documento
Uso y justificación de asociación, agregación y composición	25 %	Actividades 1 y 2, incluido el supuesto documentado
Herencia correctamente justificada	15 %	Actividad 3, preguntas 4 a 7
Interfaz y contrato de comportamiento	15 %	Actividad 4, puntos 8 a 11
Coherencia del diagrama de clases y multiplicidades	15 %	Actividades 2 y 3 y el diagrama consolidado
Implementación Java representativa y funcional	15 %	Actividad 5, apartados 5.1 a 5.3 con evidencia de ejecución
Vista funcional mediante componentes UML	10 %	Actividad 6
Argumentación técnica y claridad	5 %	Todo el documento y la conclusión
TOTAL	100 %	

Anexo – Código PlantUML de los diagramas

Código fuente de los tres diagramas para generarlos en Visual Paradigm, Draw.io o cualquier editor compatible con PlantUML. Cada bloque se pega completo, incluidas las líneas `@startuml` y `@enduml`.

A.1 Diagrama de clases completo de SmartLibrary

```
@startuml SmartLibrary_DiagramaClases

skinparam classAttributeIconSize 0
skinparam linetype ortho
skinparam shadowing false
hide circle

interface Notificable {
    + notificar(mensaje : String) : void
}

abstract class Usuario {
    - identificacion : String
    - nombre : String
    - correo : String
    + getIdentificacion() : String
    + getNombre() : String
    + getCorreo() : String
    + {abstract} descripcionRol() : String
}

class Estudiante {
    - codigoEstudiantil : String
    - programaAcademico : String
    + descripcionRol() : String
    + notificar(mensaje : String) : void
}

class Bibliotecario {
    - codigoEmpleado : String
    - turno : String
    + descripcionRol() : String
    + notificar(mensaje : String) : void
}
```

```

class Libro {
    - isbn : String
    - titulo : String
    - autor : String
    + registrarEjemplar(codigoBarras : String) : Ejemplar
    + getEjemplares() : List
}

```

```

class Ejemplar {
    - codigoBarras : String
    - disponible : boolean
    + estaDisponible() : boolean
    + marcarPrestado() : void
    + marcarDisponible() : void
}

```

```

class Prestamo {
    - fechaPrestamo : LocalDate
    - fechaPrevistaDevolucion : LocalDate
    + renovar(nuevaFecha : LocalDate) : void
    + cantidadRenovaciones() : int
}

```

```

class Renovacion {
    - fechaRenovacion : LocalDate
    - fechaAnterior : LocalDate
    - nuevaFecha : LocalDate
}

```

' ----- Herencia (es-un) -----

Usuario <|-- Estudiante

Usuario <|-- Bibliotecario

' ----- Realizacion de la interfaz -----

Notificable <|.. Estudiante

Notificable <|.. Bibliotecario

' ----- Composicion -----

Libro "1" *-- "0..*" Ejemplar : contiene >

Prestamo "1" *-- "0..*" Renovacion : registra >

```

' ----- Asociacion -----
Estudiante "1"    -- "0..*" Prestamo : solicita >
Prestamo  "0..*" --> "1"    Ejemplar : presta >

@enduml

```

A.2 Jerarquía de usuarios y contrato Notificable

```

@startuml SmartLibrary_HerenciaEInterfaz

skinparam classAttributeIconSize 0
skinparam shadowing false
hide circle

interface Notificable {
    + notificar(mensaje : String) : void
}

abstract class Usuario {
    - identificacion : String
    - nombre : String
    - correo : String
    + getNombre() : String
    + {abstract} descripcionRol() : String
}

class Estudiante {
    - codigoEstudiantil : String
    - programaAcademico : String
    + descripcionRol() : String
    + notificar(mensaje : String) : void
}

class Bibliotecario {
    - codigoEmpleado : String
    - turno : String
    + descripcionRol() : String
    + notificar(mensaje : String) : void
}

```

```

Usuario <|-- Estudiante
Usuario <|-- Bibliotecario

Notificable <|.. Estudiante
Notificable <|.. Bibliotecario

note right of Notificable
    R13: solo ALGUNOS usuarios
    reciben notificaciones.
    Por eso el metodo NO se declara
    en la clase Usuario.
end note

note bottom of Usuario
    Abstracta: en SmartLibrary
    nadie es "solo usuario".
end note

@enduml

```

A.3 Diagrama de componentes (vista funcional)

```

@startuml SmartLibrary_Componentes

skinparam componentStyle uml2
skinparam shadowing false

component "Gestion de Usuarios" as GU
component "Gestion de Catalogo" as GC
component "Gestion de Prestamos" as GP
component "Gestion de Reservas" as GR

interface "IUsuarios" as IU
interface "ICatalogo" as IC
interface "IPrestamos" as IP

' ----- Interfaces PROVISTAS -----
GU -- IU
GC -- IC
GP -- IP

```

```
' ----- Interfaces REQUERIDAS -----
```

```
GP ..> IU : <<requiere>>
```

```
GP ..> IC : <<requiere>>
```

```
GR ..> IU : <<requiere>>
```

```
GR ..> IC : <<requiere>>
```

```
GR ..> IP : <<requiere>>
```

```
note bottom of GP
```

```
    Requiere IUusuarios para validar
```

```
    al estudiante y notificarlo.
```

```
    Requiere ICatalogo para verificar
```

```
    disponibilidad del ejemplar.
```

```
end note
```

```
@endum1
```